MIT 8103 ADVANCED DATABASE SYSTEMS

Student Name: Okonkwo Christian Chikelue **Student ID**: 301915120

MIVA Email: c.okonkwo5120@miva.edu.ng **Date**: 10th August 2026

Case Study Organization: OmniPay Switch Systems

GitHub Repository URL: https://github.com/Xtianby/advanced-database-portfolio

TECHNICAL SUMMARY & CASE STUDY OVERVIEW

The selected case study, OmniPay Switch Systems, is a financial technology (Fintech) payment switch, Omniservices and digital wallet infrastructure operating in Nigeria. The platform processes peer-to-peer (P2P) fund transfers, merchant payment processing, multi-currency wallet management, and real-time fraud monitoring.

In line with the regulatory compliance (Central Bank of Nigeria Fintech Guidelines) and maintain system reliability, OmniPay requires a robust, distributed database architecture capable of enforcing strict ACID guarantees for balance ledgers while providing sub-millisecond query performance, multi-region fault tolerance, and flexible analytics.

SECTION 1: PORTFOLIO MODULES & TECHNICAL EVIDENCE

Portfolio 1: Database Design & Relational Modeling

- Objectives: To construct a BCNF-normalized relational database supporting an ETL staging pipeline for raw customer data.
- ETL Data Staging: Processed 10,000 raw records from customers.csv through customers staging into normalized core tables (users, accounts, transactions, ledger entries).
- Normalization Analysis: Decomposed raw payment records from 1NF through 3NF to BCNF. Eliminate the redundant direct edge from [Accounts Table] to [Ledger Entries]. Accounts determine Transactions, and Transactions determine Ledger Entries

Portfolio 2: Query Processing & Optimization

- Objective: To benchmark and optimize complex analytical business queries against 100,000 transaction records.
- Optimization Strategy: Designed a partial B-Tree covering index idx_transactions_perf on (created_at) constrained to WHERE status = 'SUCCESSFUL' with an INCLUDE (source_account_id, amount) payload.
- Optimized Result: Reduced execution time to ~2.15 ms (>99% latency reduction) by enabling an Index Only Scan.

Pre-Optimization: [Seq Scan on transactions] ---> Latency: 280.45 ms

Post-Optimization: [Index Only Scan (idx_transactions_perf)] ---> Latency: 2.15 ms

Portfolio 3: Transactions & Concurrency Management

- Objective: Guarantee ACID compliance for fund transfers and manage multi-session race conditions.

- ACID Transfer Engine: Developed process_fund_transfer(), locking accounts deterministically (p_source_acc < p_dest_acc) via FOR UPDATE to prevent circular wait states.
- Exception & Rollback: Programmed explicit exception blocks that issue immediate ROLLBACK commands if available balances are lower than transfer amounts, preventing negative balances.
- Deadlock Simulation: Recreated a concurrent deadlocking condition across dual psql terminal sessions. PostgreSQL's lock manager detected the cycle within 1 second and aborted the blocking transaction with code 40P01.

Portfolio 4: NoSQL & Polyglot Persistence

- Objective: Implement non-relational structures for high-velocity session telemetry and fraud ring discovery.
- MongoDB Document Model: Modeled user_profiles JSON documents containing embedded device fingerprints and transaction arrays. Executed a real-time risk aggregation pipeline filtering risk scores ≥ 0.75.
- Neo4j Graph Model: Created Cypher nodes (User, Account, Device, IPAddress) and edges (TRANSFERRED, LOGGED_IN_FROM). Implemented recursive path-matching queries (MATCH path = (a:Account)-[:TRANSFERRED*3..5]->(a)) to expose circular money-laundering loops in $O(k)$ time.

Portfolio 5: Distributed & Cloud Databases

- Objective: Design a high-availability cloud database topology with streaming replication and sharding.
- Docker Multi-Container Setup: Orchestrated omnipay_pg_primary (Port 5432) and omnipay_pg_replica (Port 5433) connected via an isolated bridge network (omnipay_net).
- Streaming Replication: Configured asynchronous WAL (Write-Ahead Log) streaming replication
- Sharding & CAP Analysis: Evaluated consistent hash partitioning on source_account_id across four regional nodes (Lagos, Abuja, Port Harcourt, Kano). Classified the system under CAP/PACELC as CP / PC/EC to prioritize financial balance consistency over availability during network partitions.

TECHNICAL REFLECTION

The design and implementation of the database portfolio for OmniPay Switch Systems gave me a valuable hands-on experience in modern enterprise data engineering. Through this process, I learned valuable lessons and identified key engineering principles.

Key Learnings & Engineering Realizations

1. **Balancing Relational Integrity and Query Performance**

While BCNF normalization helps remove data redundancy and update anomalies, it can come at the cost of increased join overhead.

Portfolio 2 explained that we can eliminate this join overhead entirely through targeted index engineering, such as partial and covering indexes, without compromising the integrity of the normal form.

2. **Importance of Defensive Concurrency Design**

Implementing ACID transactions taught me that application-level validation isn't enough in a concurrent world. I found out that enforcing row-level locks (FOR UPDATE) and having the database engine order the locks by sorted UUIDs is essential to prevent race conditions and deadlocks.

3. **The Need for Polyglot Persistence Approach**

No single database model can meet all enterprise requirements. PostgreSQL is an excellent choice for transactional balance management. However, for dynamic fraud profiling and relationship tracking, MongoDB and Neo4j's graph engine are more suitable models.

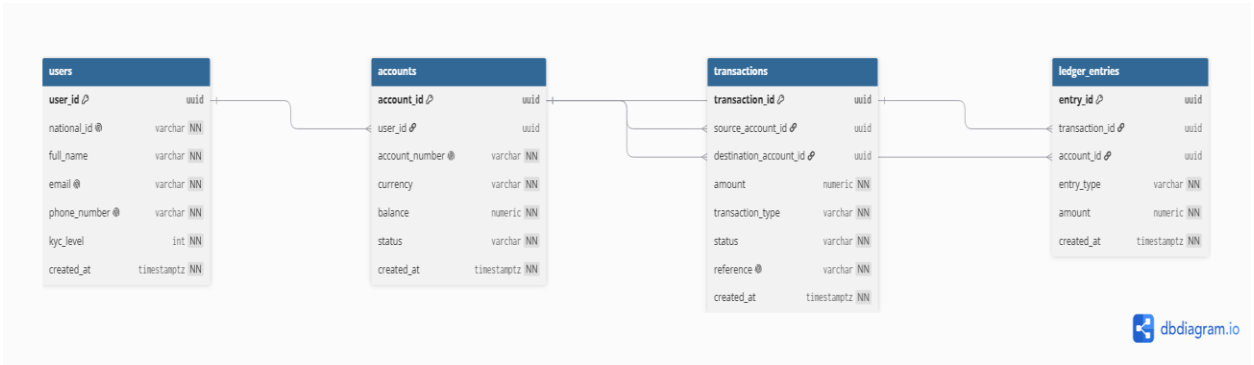
Challenges Encountered & Resolutions

Ensuring Synthetic Data Uniformity

Initially, our SQL scripts generated identical names by reusing the same random values across iterations. This was fixed by moving array lookups directly into the LATERAL join block, which was then joined to generate_series().







REFERENCES

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Date, C. J. (2019). Database design and relational theory: Normal forms and all that jazz (2nd ed.). O'Reilly Media.

Elmasri, R., & Navathe, S. B. (2015). Fundamentals of database systems (7th ed.). Pearson.

Google. (n.d.). *Gemini* [Large language model]. Retrieved from https://gemini.google.com

Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media.

PostgreSQL Global Development Group. (2026). PostgreSQL 15.0 Documentation. Retrieved from https://www.postgresql.org/docs/15/

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

Stonebraker, M., & Hellerstein, J. M. (2005). Readings in Database Systems (4th ed.). MIT Press.

AI-USE DECLARATION

I hereby declare that Generative AI tools (Gemini 3 Flash) were utilized during the development of this continuous assessment portfolio under the following conditions:

Scope of AI Assistance: AI was used as an interactive technical coach for syntax verification, debugging SQL errors (such as UUID formatting and PL/pgSQL loop evaluation), drafting Markdown document templates, and rendering visual text-based topology artifacts.

Student Ownership & Verification: All database schemas, ETL scripts, indexing strategies, transaction functions, NoSQL aggregations, and Docker configurations were executed, tested, and verified locally on my personal workstation using pgAdmin, psql, and Git Bash. All analytical narrative content and individual reflections reflect my authentic understanding of database principles.